

User Stories

US-001 User Logout Functionality

As an authenticated user, I want to click the logout option in the navigation bar, so that my session and credentials are cleared and I am safely logged out.

Description: Triggered via `NavbarComponent.logout()` calling `AuthService.logout()`, removing the JWT token from `localStorage`, clearing current user information, updating the authentication state, and redirecting the user to the Login page via Angular Router.

Acceptance Criteria:

- AC-001 When the user clicks the Logout button in the Navbar, the system must trigger `NavbarComponent.logout()` which calls `AuthService.logout()`. [Source: BRD §4.3.1.1, step 1]
- AC-002 The `AuthService` must remove the JWT token from `localStorage`. [Source: BRD §4.3.1.1, step 1]
- AC-003 The `AuthService` must clear current user information and update the authentication state (e.g., via session `BehaviorSubject/Observable`). [Source: BRD §4.3.1.1, step 1]
- AC-004 Upon successful logout and state update, the Angular Router must navigate the user to the Login page. [Source: BRD §4.3.1.1, step 1]
- AC-005 Once the token is removed, the user must no longer be treated as authenticated. [Source: BRD §4.3.1.1, step 1]

Business Rules:

- BR-001 User session and credentials must be cleared and the user must no longer be treated as authenticated once the token is removed.

Dependencies:

- `NavbarComponent`, `AuthService`, and Angular Router.

Assumptions:

- Not specified in the provided input.

Edge Cases:

- Not specified in the provided input.

US-002 Authentication Guard for Protected Pages

As a system administrator, I want protected pages to be guarded by an authentication guard, so that unauthenticated users are blocked and redirected.

Description: Execute the `canActivate` guard prior to loading protected route components (e.g., dashboard) to check authentication status via `AuthService` and allow or block/redirect navigation.

Acceptance Criteria:

- AC-001 When a user attempts to navigate to a protected route (e.g., dashboard), the system must execute the `canActivate` guard prior to loading the component. [Source: BRD §4.3.1.1, step 1]
- AC-002 The `AuthGuard` must check the user's authentication status using `AuthService`. [Source: BRD §4.3.1.1, step 1]
- AC-003 If the user is authenticated, the guard must allow navigation to the requested component. [Source: BRD §4.3.1.1, step 1]
- AC-004 If the user is not authenticated, the guard must block navigation and/or redirect the user away from the protected route. [Source: BRD §4.3.1.1, step 1]

Business Rules:

- BR-001 Unauthenticated users must be blocked and redirected when attempting to access protected routes.

Dependencies:

- AuthGuard and AuthService.

Assumptions:

- Not specified in the provided input.

Edge Cases:

- Not specified in the provided input.

US-003 HTTP Interceptor for Authentication Token

As an API client/developer, I want an HTTP Interceptor to automatically attach the authentication token to outgoing requests, so that backend APIs can verify user identity seamlessly.

Description: AuthInterceptor intercepts outgoing HTTP requests dispatched through Angular HttpClient to retrieve and attach the authentication token to the request payload/headers sent to backend APIs. Configured globally using provideHttpClient() and providers array.

Acceptance Criteria:

- AC-001 The AuthInterceptor must intercept outgoing HTTP requests dispatched through Angular HttpClient. [Source: BRD §4.3.1.1, step 1]
- AC-002 The interceptor must retrieve the authentication token and attach it to the outgoing request payload/headers sent to backend APIs. [Source: BRD §4.3.1.1, step 1]
- AC-003 The HTTP Interceptor and HttpClient must be configured globally in the application via provideHttpClient() and providers array. [Source: BRD §4.3.1.1, step 1]

Business Rules:

- BR-001 Authentication token must be attached to outgoing requests so backend APIs can verify user identity.

Dependencies:

- AuthInterceptor, Angular HttpClient, provideHttpClient().

Assumptions:

- Not specified in the provided input.

Edge Cases:

- Not specified in the provided input.

US-004 Responsive Navigation Bar with Dynamic Session Routing

As a user, I want a responsive navigation bar that indicates active routes and displays session-dependent navigation options, so that I can easily navigate the application across different devices.

Description: Navigation bar expands on large screens and collapses on smaller screens using Bootstrap classes (navbar-expand-lg). Navigation items use RouterLink and RouterLinkActive with exact match option. Template listens to authService.session\$ via async pipe and includes styling for brand/logo, dark background theme (bg-dark, navbar-dark), bottom margin spacing (mb-4), and shadow (shadow-sm).

Acceptance Criteria:

- AC-001 The navigation bar must expand on large screens and collapse into a mobile menu on smaller screens using Bootstrap classes (navbar-expand-lg). [Grounding: DERIVED]
- AC-002 The navigation items must use RouterLink for single-page component routing and RouterLinkActive with exact match option to highlight the active route. [Grounding: DERIVED]
- AC-003 The navigation template must listen to authService.session\$ via async pipe to dynamically render session status and display relevant links or logout button. [Grounding: DERIVED]
- AC-004 The navigation bar must include styling for brand/logo, dark background theme (bg-dark, navbar-dark), bottom margin spacing (mb-4), and shadow (shadow-sm). [Grounding: DERIVED]

Business Rules:

- BR-001 Navigation options must dynamically render based on session status using `authService.session$` via async pipe.

Dependencies:

- Bootstrap framework, RouterLink, RouterLinkActive, `authService.session$`.

Assumptions:

- Not specified in the provided input.

Edge Cases:

- Not specified in the provided input.